Solidity & EVM Interview Q&A

Printable version (generated 2026-01-30)

Notes: Some answers depend on the chain's active hard-fork rules and on Solidity/compiler version. For deployment limits and protocol semantics, the canonical references are the relevant EIPs and Solidity docs.

Easy

1. What is the difference between private, internal, public, and external functions?

Answer: `private` = callable only inside the defining contract. `internal` = callable inside the contract + derived contracts (and libraries via internal). `public` = callable internally and externally. `external` = callable only from outside via ABI (but can still be invoked internally as `this.f()` which becomes an external call).

2. Approximately, how large can a smart contract be?

Answer: On Ethereum mainnet, deployed runtime bytecode is capped at 24,576 bytes (0x6000) (EIP-170). The contract creation initcode is capped at 49,152 bytes (0xC000) (EIP-3860). (Other chains/forks may differ.)

3. What is the difference between create and create2?

Answer: `CREATE` address depends on deployer address + deployer nonce. `CREATE2` address is deterministic from deployer + salt + initcode hash (so you can precompute it).

4. What major change with arithmetic happened with Solidity 0.8.0?

Answer: Overflow/underflow checks are enabled by default and will revert (unless inside `unchecked { ... }`).

5. What special CALL is required for proxies to work?

Answer: `DELEGATECALL` (execute implementation code in the proxy's storage/balance context).

6. How do you calculate the dollar cost of an Ethereum transaction?

Answer: `costETH = gasUsed * (baseFeePerGas + priorityFeePerGas)`; then `costUSD = costETH * ETHUSD`. (Legacy tx uses `gasPrice` instead of base+tip.)

7. What are the challenges of creating a random number on the blockchain?

Answer: Anything derived from on-chain/public data can be predicted or manipulated by transaction ordering and, to some extent, block producers. Robust randomness needs commit-reveal, VRF, or a consensus-level randomness beacon.

8. What is the difference between a Dutch Auction and an English Auction?

Answer: Dutch: price starts high and decreases until someone buys. English: price starts low and increases via bids until the highest bid wins.

9. What is the difference between transfer and transferFrom in ERC20?

Answer: `transfer` moves tokens from `msg.sender`. `transferFrom` moves tokens from another address using an allowance (spender pattern).

10. Which is better to use for an address allowlist: a mapping or an array? Why?

Answer: Usually a `mapping(address => bool)` for O(1) membership checks and gas predictability. Arrays require O(n) scans unless you add indexing structures.

11. Why shouldn't tx.origin be used for authentication?

Answer: Because it's phishable: a malicious contract can trick a user into calling it, and then it calls your contract; `tx.origin` stays the user.

12. What hash function does Ethereum primarily use?

Answer: Keccak-256.

13. How much is 1 gwei of Ether?

Answer: 1 gwei = 10^9 wei = 10^-9 ETH.

14. How much is 1 wei of Ether?

Answer: 1 wei = 10^-18 ETH.

15. What is the difference between assert and require?

Answer: `require` is for input/expected-condition checks; `assert` is for invariants and triggers a Panic on failure.

16. What is a flash loan?

Answer: An uncollateralized loan that must be borrowed and repaid within the same transaction, otherwise the whole transaction reverts.

17. What is the check-effects-interaction pattern?

Answer: Do checks first, then update effects (state), and only then do interactions (external calls), to reduce reentrancy risk.

18. What is the minimum amount of Ether required to run a solo staking node?

Answer: 32 ETH per validator.

19. What is the difference between fallback and receive?

Answer: `receive()` runs on plain ETH transfers with empty calldata (if present). `fallback()` runs when no function matches or when calldata is non-empty; it can also be `payable`.

20. What is reentrancy?

Answer: When an external call lets an attacker re-enter your contract before you've finished updating state.

21. What prevents infinite loops from running forever?

Answer: The gas limit.

22. What is the difference between tx.origin and msg.sender?

Answer: `tx.origin` is the original EOA that started the transaction; `msg.sender` is the immediate caller.

23. How do you send Ether to a contract that does not have payable functions, or a receive or fallback?

Answer: You can force-send ETH via `SELFDESTRUCT` from another contract.

24. What is the difference between view and pure?

Answer: `view` may read state but not modify it; `pure` may not read or modify state.

25. What is the difference between transferFrom and safeTransferFrom in ERC721?

Answer: `safeTransferFrom` checks contract recipients via `onERC721Received`; `transferFrom` does not.

26. How can an ERC1155 token be made into a non-fungible token?

Answer: Treat each `id` as unique and enforce supply = 1 (or NF-class semantics per id).

27. What is access control and why is it important?

Answer: Restricting who can call sensitive functions (mint, upgrade, withdraw). Without it, anyone can take privileged actions.

28. What does a modifier do?

Answer: Wraps a function with pre/post logic around the _ placeholder.

29. What is the largest value a uint256 can store?

Answer: `2^256 - 1`.

30. What is variable and fixed interest rate?

Answer: Variable changes over time based on utilization/market parameters; fixed is locked (or bounded) at a predetermined rate for a period.

Medium

31. What is the difference between transfer and send? Why should they not be used?

Answer: Both forward a fixed 2300 gas stipend; `transfer` reverts on failure, `send` returns `false`. They're discouraged because stipend assumptions break; prefer `call{value: ...}("")`.

32. What is a storage collision in a proxy contract?

Answer: With `delegatecall`, implementation writes to proxy storage. If storage layouts overlap (e.g., slot 0), state/admin pointers can be corrupted.

33. What is the difference between abi.encode and abi.encodePacked?

Answer: `abi.encode` is standard ABI encoding (padded, offsets/lengths). `abi.encodePacked` is tightly packed (no padding/offsets), which can be ambiguous.

34. uint8, uint32, uint64, uint128, uint256 are all valid uint sizes. Are there others?

Answer: Yes: any multiple of 8 bits from 8 to 256 (e.g., `uint16`, `uint24`, `uint200`).

35. What changed with block.timestamp before and after proof of stake?

Answer: Under PoS, time is structured into 12-second slots; missed slots create gaps. Under PoW, times were probabilistic.

36. What is frontrunning?

Answer: Observing a pending transaction and submitting another that executes before it to profit.

37. What is a commit-reveal scheme and when would you use it?

Answer: Commit a hash of a secret, reveal later. Used to prevent others from learning your choice early (games, sealed bids, randomness).

38. Under what circumstances could abi.encodePacked create a vulnerability?

Answer: When concatenation becomes ambiguous (especially with dynamic types), allowing hash collisions and confusion.

39. How does Ethereum determine the BASEFEE in EIP-1559?

Answer: Base fee adjusts each block based on how far gasUsed is from target: >target increases, <target decreases, with a per-block cap.

40. What is the difference between a cold read and a warm read?

Answer: First access to an account/storage slot in a transaction is "cold" (more expensive); subsequent accesses are "warm" (cheaper).

41. How does an AMM price assets?

Answer: By a pricing function (e.g., constant product `x*y=k`), where marginal price derives from reserve ratios.

42. What is a function selector clash in a proxy and how does it happen?

Answer: Two signatures can share the same 4-byte selector, or proxy admin functions conflict with implementation selectors.

43. What is the effect on gas of making a function payable?

Answer: Slightly cheaper because Solidity can skip the implicit `msg.value == 0` check.

44. What is a signature replay attack?

Answer: Reusing a valid signature in a different context if the signed message lacks domain separation (chainId/contract/nonce).

45. How would you design a game of rock-paper-scissors in a smart contract such that players cannot cheat?

Answer: Commit-reveal: commit `hash(choice, salt)`; reveal later. Add timeouts/forfeits.

46. What is the free memory pointer and where is it stored?

Answer: Stored at memory slot 0x40 by Solidity convention.

47. What function modifiers are valid for interfaces? What is the difference between memory and calldata in a function argument?

Answer: Interface functions are declarations; they can be `external`, `view`, `pure`, `payable`. `calldata` is read-only external input; `memory` is mutable temporary memory.

48. Describe the three types of storage gas costs for writes.

Answer: 0→nonzero expensive; nonzero→nonzero cheaper; nonzero→0 cheaper and may yield a refund (refund rules evolved).

49. Why shouldn't upgradeable contracts use the constructor?

Answer: Constructors run only on the implementation deployment, not via proxy; proxy storage won't be initialized.

50. What is the difference between UUPS and the Transparent Upgradeable Proxy pattern?

Answer: Transparent: upgrade/admin logic in proxy. UUPS: upgrade logic in implementation; proxy is minimal.

51. If a contract delegatecalls an empty address or an implementation that was previously self-destructed, what happens? What if it is a low-level call instead of a delegatecall?

Answer: If target has no code, the call typically "succeeds" doing nothing and returns empty data; decoding expected returndata may revert. `call` runs in callee context; `delegatecall` runs in caller context.

52. What danger do ERC777 tokens pose?

Answer: Hook-based callbacks enable reentrancy in contracts that assume ERC20-like transfers.

53. According to the solidity style guide, how should functions be ordered?

Answer: Recommended order: constructor, receive, fallback, external, public, internal, private; within a group put `view`/`pure` last.

54. According to the solidity style guide, how should function modifiers be ordered?

Answer: Common convention: visibility, mutability, virtual, override, custom modifiers.

55. What is a bonding curve?

Answer: A deterministic pricing function relating token price to supply.

56. How does _safeMint differ from _mint in the OpenZeppelin ERC721 implementation? What keywords are provided in Solidity to measure time? What is a sandwich attack?

Answer: `_safeMint` checks recipient contracts via `onERC721Received`; `_mint` does not. Time units: `seconds`, `minutes`, `hours`, `days`, `weeks` (`years` deprecated). Sandwich: attacker front-runs and back-runs a victim trade to extract value.

57. If a delegatecall is made to a function that reverts, what does the delegatecall do?

Answer: It returns failure and revert data; proxies usually bubble up the revert.

58. What is a gas efficient alternative to multiplying and dividing by a power of two?

Answer: Bit shifting: x<<k and x>>k (with signed-care).

59. How large a uint can be packed with an address in one slot?

Answer: Address is 20 bytes, leaving 12 bytes: up to uint96.

60. Which operations give a partial refund of gas? What is ERC165 used for?

Answer: Clearing storage (SSTORE nonzero→0) yields refunds (reduced by EIP-3529). ERC165 is interface detection via `supportsInterface`.

61. If a proxy makes a delegatecall to A, and A does address(this).balance, whose balance is returned, the proxy's or A?

Answer: The proxy's balance.

62. What is a slippage parameter useful for?

Answer: It bounds execution ("min out" / "max price"), protecting against MEV and price movement.

63. What does ERC721A do to reduce mint costs? What is the tradeoff?

Answer: Batch mints with sparse ownership storage. Tradeoff: more complex ownership logic; some transfers/queries can cost more.

64. Why doesn't Solidity support floating point arithmetic?

Answer: Determinism and complexity; floating point introduces rounding hazards and expensive emulation.

65. What is TWAP?

Answer: Time-weighted average price.

66. How does Compound Finance calculate utilization?

Answer: Classic formula: `borrows / (cash + borrows - reserves)`.

67. If a delegatecall is made to a function that reads from an immutable variable, what will the value be?

Answer: The immutable is baked into implementation code, so it reads the value from the implementation deployment.

68. What is a fee-on-transfer token?

Answer: A token that charges a fee during transfers so recipient receives less than sent.

69. What is a rebasing token?

Answer: A token that periodically adjusts balances and/or total supply per a rule.

70. In what year will a timestamp stored in a uint32 overflow?

Answer: Around 2106.

71. What is LTV in the context of DeFi?

Answer: Loan-to-value: max borrow as a fraction of collateral value.

72. What are aTokens and cTokens in the context of Compound Finance and AAVE?

Answer: Interest-bearing receipt tokens representing supplied assets plus accrued interest.

73. Describe how to use a lending protocol to go leveraged long or leveraged short on an asset.

Answer: Long: borrow stable, buy asset, resupply, loop. Short: borrow asset, sell to stable, resupply stable, loop; unwind by buying back.

74. What is a perpetual protocol?

Answer: Derivatives protocol offering perpetual swaps (no expiry) with funding payments.

Hard

75. How does fixed point arithmetic represent numbers?

Answer: Store integers scaled by a factor (e.g., 1e18). Real value = stored/scale.

76. What is an ERC20 approval frontrunning attack?

Answer: Changing allowance X→Y can be raced: spender front-runs and spends X before new approval lands.

77. What opcode accomplishes address(this).balance?

Answer: `SELFBALANCE` (or `BALANCE` depending on compiler).

78. How many arguments can a solidity event have?

Answer: No strict small number limit; but only up to 3 indexed params for non-anonymous events (4 topics total including signature).

79. What is an anonymous Solidity event?

Answer: An event that omits the signature topic, allowing 4 indexed params.

80. Under what circumstances can a function receive a mapping as an argument?

Answer: Only as a storage reference in internal/library functions; mappings can't be ABI-encoded for external calls.

81. What is an inflation attack in ERC4626

Answer: In nearly empty vaults, donation/rounding can skew share price so later depositors receive fewer shares.

82. How many storage slots does this use? uint64[] x = [1,2,3,4,5]? Does it differ from memory?

Answer: Storage: 1 slot length + packed elements (4 per slot) → 2 slots data = 3 total. Memory: each element is 32 bytes.

83. Prior to the Shanghai upgrade, under what circumstances is `RETURNDATASIZE` more efficient than pushing literal 0?

Answer: Pre-PUSH0, `RETURNDATASIZE` is a 1-byte opcode that often yields 0 early in execution, while `PUSH1 0x00` is 2 bytes. It is only safe as a "push zero" substitute when you know the last call's return data size is 0 (e.g., before any external calls, or after a call you know returned empty data).

84. Why does the compiler insert the INVALID op code into Solidity contracts?

Answer: As an unreachable trap (e.g., "should never happen" paths) and to hard-fail if reached.

85. What is the difference between how a custom error and a require with error string is encoded at the EVM level?

Answer: `require("msg")` uses `Error(string)` with big string encoding. Custom errors use a 4-byte selector + encoded args, usually much smaller.

86. What is the kink parameter in the Compound DeFi formula?

Answer: A utilization threshold where interest slope increases sharply ("jump rate model").

87. How can the name of a function affect its gas cost, if at all?

Answer: Runtime gas: name doesn't matter; selector matters. Deployment can change slightly due to bytecode differences.

88. What is a common vulnerability with ecrecover?

Answer: Not checking for `address(0)` result, signature malleability (high-s), and replay due to missing domain separation.

89. What is the difference between an optimistic rollup and a zk-rollup?

Answer: Optimistic uses fraud proofs + challenge window; zk uses validity proofs.

90. How does EIP1967 pick the storage slots, how many are there, and what do they represent?

Answer: Slots are `keccak256("eip1967.proxy.X") - 1` to avoid collisions. Core: implementation, admin, beacon.

91. How much is one Sazbo of ether?

Answer: Interpreting as szabo: `10^12 wei = 10^-6 ETH`.

92. What can delegatecall be used for besides use in a proxy?

Answer: Plugin/module architectures, diamond routing, shared logic with one storage domain.

93. Under what circumstances would a smart contract that works on Etheruem not work on Polygon or Optimism? (Assume no dependencies on external contracts)

Answer: Differences in chain rules/params: chainId, opcode availability/EVM version, gas schedule, block timing/finality, and L2 semantics.

94. How can a smart contract change its bytecode without changing its address?

Answer: Use a proxy (address constant, implementation changes). Metamorphic CREATE2+SELFDESTRUCT patterns existed but are restricted post-EIP-6780.

95. What is the danger of putting msg.value inside of a loop?

Answer: You may unintentionally reuse the same ETH amount multiple times or create cumulative sends exceeding balance.

96. Describe the calldata of a function that takes a dynamic length array of uint128 when [1,2,3,4] is passed as an argument

Answer: selector + head offset; then length=4; then 4 elements each as 32-byte word containing the uint128 value.

97. Why is strict inequality comparisons more gas efficient than ≤ or ≥? What extra opcode(s) are added?

Answer: EVM has `LT/GT`. Non-strict uses `LT/GT` plus `ISZERO` (or similar) so adds at least one opcode.

98. If a proxy calls an implementation, and the implementation self-destructs in the function that gets called, what happens?

Answer: Via `delegatecall`, SELFDESTRUCT executes in proxy context: it can sweep ETH; code deletion is restricted post-EIP-6780.

99. What is the relationship between variable scope and stack depth?

Answer: More simultaneously-live locals/temps increases stack pressure; narrowing scopes reduces “stack too deep”.

100. What is an access list transaction?

Answer: A transaction type that includes addresses+storage keys to pre-warm and reduce cold access costs.

101. How can you halt an execution with the mload opcode?

Answer: `mload` at a huge offset forces massive memory expansion cost causing out-of-gas.

102. What is a beacon in the context of proxies?

Answer: A contract holding implementation address; proxies reference it so upgrading the beacon upgrades all proxies.

103. Why is it necessary to take a snapshot of balances before conducting a governance vote?

Answer: Prevent vote manipulation via transfers/flash loans during voting.

104. How can a transaction be executed without a user paying for gas?

Answer: Meta-transactions / account abstraction: a relayer/paymaster pays gas with user authorization.

105. In solidity, without assembly, how do you get the function selector of the calldata?

Answer: `msg.sig` or `bytes4(msg.data)`.

106. How is an Ethereum address derived?

Answer: EOA: last 20 bytes of Keccak-256 of public key. CREATE: last 20 bytes of Keccak-256(RLP(sender, nonce)). CREATE2: per CREATE2 formula.

107. What is the metaproxy standard?

Answer: ERC-3448 MetaProxy: minimal proxy that appends immutable metadata to bytecode.

108. If a try catch makes a call to a contract that does not revert, but a revert happens inside the try block, what happens?

Answer: `try/catch` only catches failures of the external call; a revert in your own code inside `try {}` is not caught.

109. If a user calls a proxy makes a delegatecall to A, and A makes a regular call to B, from A's perspective, who is msg.sender? from B's perspective, who is msg.sender? From the proxy's perspective, who is msg.sender?

Answer: Proxy: user. A (delegatecalled): user. B (called): proxy.

110. Under what circumstances do vanity addresses (leading zero addresses) save gas?

Answer: Only when embedded as constants: fewer nonzero bytes can shorten PUSH and reduce bytecode size.

111. Why do a significant number of contract bytecodes begin with 6080604052? What does that bytecode sequence do?

Answer: `PUSH1 0x80; PUSH1 0x40; MSTORE` sets free-memory pointer at 0x40 to 0x80.

112. How does Uniswap V3 determine the boundaries of liquidity intervals?

Answer: Positions specify lower/upper ticks aligned to `tickSpacing`. Liquidity is active only within range.

113. What is the risk-free rate?

Answer: The theoretical return with zero default risk (often proxied by government bills in TradFi).

114. When a contract calls another via call, delegatecall, or staticcall, how is information passed between them?

Answer: Inputs via calldata (plus value/gas). Outputs via success flag + returndata. Context differs across call types.

115. What is the difference between bytes and bytes1[]?

Answer: `bytes` is packed contiguous bytes; `bytes1[]` is an array with overhead and typically less efficient.

116. What is the most amount of leverage that can be achieved in a borrow-swap-supply-collateral loop if the LTV is 75%? What about other LTV limits?

Answer: Approx max leverage = `1/(1-LTV)`. For 75%: 4x. 50%:2x. 80%:5x. 90%:10x.

117. How does Curve StableSwap achieve concentrated liquidity?

Answer: StableSwap invariant with amplification factor A flattens curve near peg (more liquidity around 1:1).

118. What quirks does the Tether stablecoin contract have?

Answer: Generally: centralized admin controls like freezing/blacklisting and mint/burn control; specifics vary by deployment/version.

119. What is the smallest uint that will store 1 million? 1 billion? 1 trillion? 1 quadrillion?

Answer: 1 million -> `uint24`. 1 billion -> `uint32`. 1 trillion -> `uint40`. 1 quadrillion -> `uint56`.

120. What danger to uninitialized UUPS logic contracts pose?

Answer: If implementation isn't initialized/locked, attacker can initialize it and become upgrader/owner, potentially bricking/abusing upgrade flows.

121. What is the difference (if any) between what a contract returns if a divide-by-zero happens in Solidity or if a divide-by-zero happens in Yul?

Answer: Solidity reverts with a Panic. In raw Yul/EVM, `DIV` by zero yields 0 unless you add checks.

122. Why can't .push() be used to append to an array in memory?

Answer: Memory arrays are fixed-size once allocated; `.push()` is only for storage dynamic arrays.

Advanced

123. What addresses do the ethereum precompiles live at?

Answer: Precompiles live at `0x01`–`0x09`, and with EIP-4844 there is also `0x0A` for point evaluation.

124. Describe what “liquidity” is in the context of Uniswap V2 and Uniswap V3.

Answer: V2: LP share of full-range pool. V3: liquidity is range-bound; capital concentrated around chosen prices.

125. If a delegatecall is made to a contract that makes a delegatecall to another contract, who is msg.sender in the proxy, the first contract, and the second contract?

Answer: All see the original external caller as `msg.sender`.

126. What is the difference between how a uint64 and uint256 are abi-encoded in calldata?

Answer: Both occupy 32 bytes when passed as arguments; uint64 is left-padded.

127. What is read-only reentrancy?

Answer: Reentrancy that exploits inconsistent reads during external calls (e.g., oracle/price reads mid-state).

128. What are the security considerations of reading a (memory) bytes array from an untrusted smart contract call?

Answer: Returned data can be huge (DoS), malformed for decoding, or crafted to break assumptions; cap sizes and handle decode failures.

129. If you deploy an empty Solidity contract, what bytecode will be present on the blockchain, if any?

Answer: Yes, there is still deployed runtime bytecode (often minimal) plus metadata.

130. How does the EVM price memory usage?

Answer: Memory expansion cost increases with the highest accessed word, with linear + quadratic components.

131. What is stored in the metadata section of a smart contract?

Answer: CBOR-encoded compiler metadata (compiler version and source/settings references, often IPFS/Swarm hash).

132. What is the uncle-block attack from an MEV perspective?

Answer: PoW-era withholding/reorg strategies could capture MEV or rewards. Post-Merge, “uncles” aren’t part of block production the same way.

133. How do you conduct a signature malleability attack?

Answer: Exploit ECDSA malleability: `(r, s)` and `(r, n-s)` if low-s isn’t enforced.

134. Under what circumstances do addresses with leading zeros save gas and why?

Answer: Same as vanity: if hardcoded, smaller PUSH/bytecode.

135. What is the difference between payable(msg.sender).call{value: value}("") and msg.sender.call{value: value}("")?

Answer: EVM-wise none; Solidity requires an `address payable` to send value, so `payable(msg.sender)` is an explicit cast.

136. How many storage slots does a string take up?

Answer: <=31 bytes uses 1 slot packed. Longer stores length in slot and data at `keccak256(slot)` across multiple slots.

137. How does the --via-ir functionality in the Solidity compiler work?

Answer: Compiles via Yul IR pipeline (different codegen/optimizations), can change bytecode and fix stack-too-deep.

138. Are function modifiers called from right to left or left to right, or is it non-deterministic?

Answer: Deterministic left-to-right; leftmost is outermost.

139. If you do a delegatecall to a contract and the opcode CODESIZE executes, which contract size will be returned?

Answer: The size of the currently executing code (the implementation code).

140. Why is it important to ECDSA sign a hash rather than an arbitrary bytes32?

Answer: To ensure structured domain separation and avoid ambiguous interpretations; EIP-191/EIP-712 patterns.

141. Describe how symbolic manipulation testing works.

Answer: Symbolic execution explores paths with symbolic inputs; SMT solvers find concrete counterexamples that violate invariants.

142. What is the most efficient way to copy regions of memory?

Answer: Use `MCOPY` where available; otherwise word-wise `mload/mstore` or `calldatacopy/returndatacopy` as appropriate.

143. How can you validate on-chain that another smart contract emitted an event, without using an oracle?

Answer: You generally can't read historical logs from EVM state. You'd need state commitments or verify receipts proofs supplied externally.

144. When selfdestruct is called, at what point is the Ether transferred? At what point is the smart contract's bytecode erased?

Answer: Balance is swept as part of execution semantics; state is finalized end-of-tx. Code deletion is restricted post-EIP-6780 (generally only same-tx created contracts).

145. Under what conditions does the Openzeppelin Proxy.sol overwrite the free memory pointer? Why is it safe to do this?

Answer: It uses memory from 0x00 to copy calldata/returndata, overwriting 0x40. Safe because it returns/reverts directly from assembly without resuming Solidity.

146. Why did Solidity deprecate the "years" keyword?

Answer: Because "years" is ambiguous (leap years), so it can mislead.

147. What does the verbatim keyword do, and where can it be used?

Answer: Inserts exact code in inline assembly/Yul with explicit stack I/O (expert-only).

148. How much gas can be forwarded in a call to another smart contract?

Answer: Up to `gasleft()` but capped by EIP-150's 63/64 rule.

149. What does an int256 variable that stores -1 look like in hex?

Answer: `0x` followed by 64 `f` characters.

150. What is the use of the signextend opcode?

Answer: Sign-extends a smaller signed integer to 256 bits.

151. Why do negative numbers in calldata cost more gas?

Answer: Calldata charges more for nonzero bytes; negatives have leading 0xff bytes in two's complement.

152. What is a zk-friendly hash function and how does it differ from a non-zk-friendly hash function?

Answer: ZK-friendly hashes are efficient in arithmetic circuits (Poseidon/MiMC); Keccak/SHA are bit-oriented and expensive in circuits.

153. What does a metaproxy do?

Answer: A proxy/cloning pattern that embeds immutable metadata in bytecode to parameterize behavior.

154. What is a nullifier in the context of zero knowledge, and what is it used for?

Answer: A unique derived value to prevent double-spends while preserving privacy.

155. What is SECP256K1?

Answer: The elliptic curve used by Ethereum for ECDSA signatures.

156. Why shouldn't you get price from slot0 in Uniswap V3?

Answer: `slot0` is instantaneous and manipulable within a tx; use TWAP observations for oracle-like use.

157. Describe how to compute the 9th root of a number on-chain in Solidity.

Answer: Use fixed-point and iterative approximation (e.g., Newton-Raphson on `x^9 - a`).

158. What is the danger of using return in assembly out of a Solidity function that has a modifier?

Answer: It can bypass modifier post-logic (cleanup/unlocks), breaking invariants.

159. Without using the % operator, how can you determine if a number is even or odd?

Answer: `x & 1 == 0` means even.

160. What does codesize() return if called within the constructor? What about outside the constructor?

Answer: `CODESIZE` returns the size of currently executing code: initcode in constructor, runtime code after deployment. `EXTCODESIZE(address(this))` is 0 during construction.

Key references (non-exhaustive)

Solidity documentation: https://docs.soliditylang.org/
Solidity Style Guide: https://docs.soliditylang.org/en/latest/style-guide.html
EIP-170 (contract runtime code size limit): https://eips.ethereum.org/EIPS/eip-170
EIP-3860 (initcode size limit & metering): https://eips.ethereum.org/EIPS/eip-3860
EIP-1559 (base fee mechanism): https://eips.ethereum.org/EIPS/eip-1559
EIP-1967 (proxy storage slots): https://eips.ethereum.org/EIPS/eip-1967
ERC-3448 (MetaProxy standard): https://eips.ethereum.org/EIPS/eip-3448
EIP-4844 (KZG point evaluation precompile @ 0x0A): https://eips.ethereum.org/EIPS/eip-4844
EIP-6780 (SELFDESTRUCT semantics change): https://eips.ethereum.org/EIPS/eip-6780